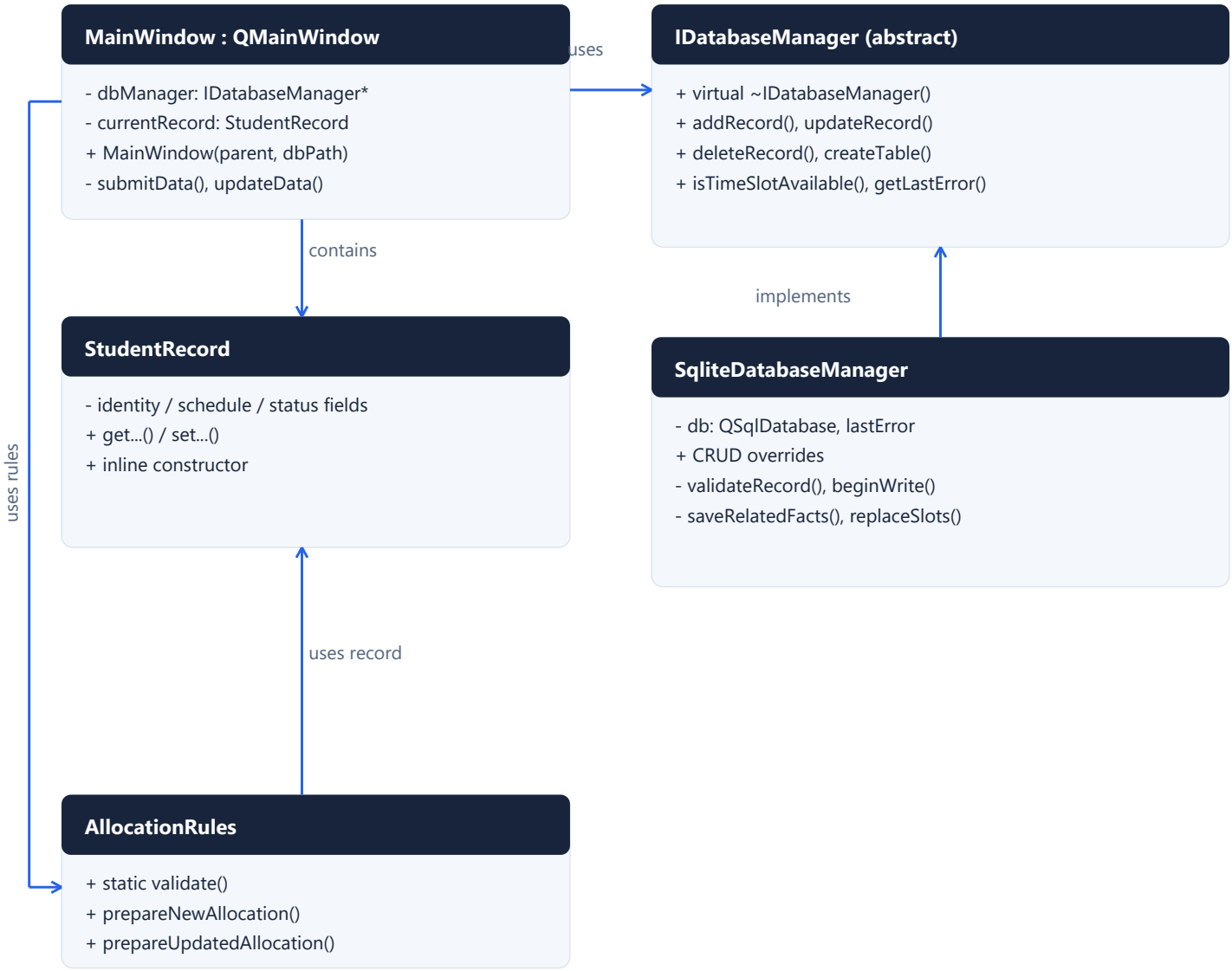
A4 / DESIGN AND MODELLING

# OOP and software design




Solid arrows labelled implements show inheritance/realization. Other labelled arrows indicate use or containment. MainWindow owns the database interface pointer and current StudentRecord value.

| Additional Qt subclass | Implemented behavior |
|---|---|
| CustomCalendarWidget : QCalendarWidget | Overrides paintCell() for status-count decoration. |
| CustomAlertDialog / CustomInputDialog : QDialog | Reusable prompts and feedback. |
| AdminSettingsDialog / SystemSettingsDialog : QDialog | Configuration validation and persistence. |

DESIGN EXPLANATION / 1 OF 2

# Design note: object boundaries

## Encapsulation

StudentRecord keeps its fields private and exposes getters/setters for identity, schedule, supervisor, project, allocation key and status. SqliteDatabaseManager keeps the QSqlDatabase connection, connection name and last error private. Validation, transaction start/commit and child-slot replacement stay inside this class.

## Inheritance

SqliteDatabaseManager derives from IDatabaseManager and implements its persistence contract. MainWindow derives from QMainWindow to participate in the Qt widget lifecycle. The calendar derives from QCalendarWidget and the custom prompts/settings derive from QDialog so they reuse Qt input and event behavior.

## Runtime polymorphism

MainWindow stores an IDatabaseManager pointer and constructs a SqliteDatabaseManager. Calls such as addRecord(), updateRecord() and isTimeSlotAvailable() dispatch through virtual methods. The interface has a virtual destructor. Qt also calls CustomCalendarWidget::paintCell() through its override when rendering date cells.

## Composition and ownership

MainWindow contains the current StudentRecord value and owns the database manager. Parent-child ownership handles widgets, while generated UI helper objects are deleted explicitly. The database manager destructor closes and removes its named Qt connection.

Evidence: models/studentrecord.h, database/idatabasemanager.h, database/databasemanager.h, ui/mainwindow.h and ui/widgets/customcalendarwidget.h.

DESIGN EXPLANATION / 2 OF 2

# Design note: rules and interaction

## Rules outside the window

AllocationRules supplies a Field enum and static validation/status preparation functions. It identifies the first input error, creates the four-digit key and timestamp for a new allocation, and resets student edits to Pending. Keeping these decisions together makes their behavior testable without navigating every screen.

## Persistence abstraction

IDatabaseManager describes operations the user interface needs. The SQLite implementation binds input values to prepared SQL statements. It groups parent/profile facts, allocation updates and slot replacement in one write transaction to avoid partial records.

## Event-driven interaction

Qt signals and slots link button clicks, calendar selection, status changes and setting signals to application behavior. This is the framework event mechanism. The database interface resembles a repository boundary, but the code does not implement a full dependency-injection framework or an interchangeable database factory.

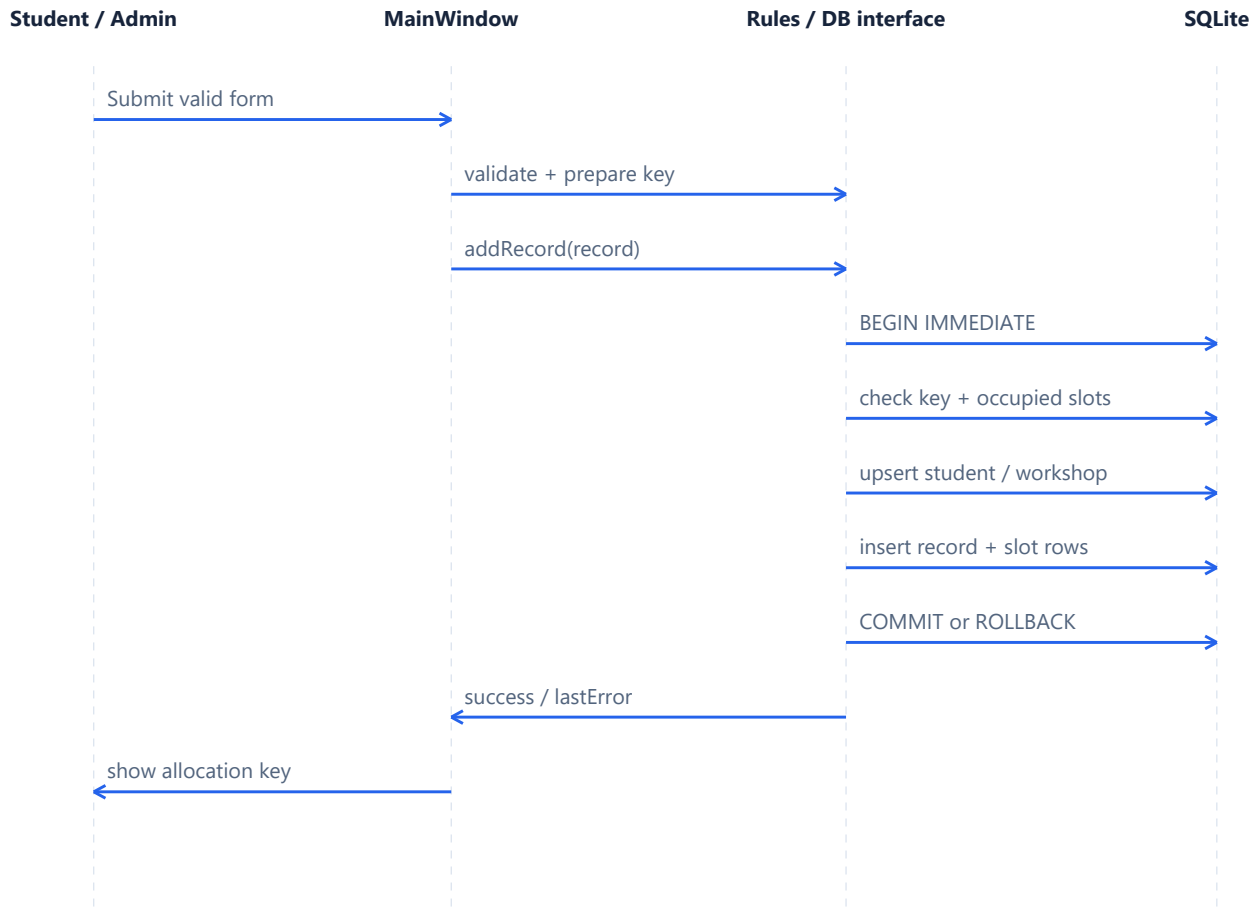
## Build decisions and limits

The .pro file compiles the monolithic mainwindow.cpp and databasemanager.cpp. Several older split .cpp files remain in the source archive but are excluded. StudentRecord methods are inline in the header. Adding excluded files to the build without reconciliation would introduce incompatible APIs or duplicate definitions.

The large MainWindow still combines navigation and orchestration. A future refactor could extract booking/admin controllers and inject the database interface while preserving the current virtual contract.

CRITICAL FLOW / 1

# Sequence: create an allocation



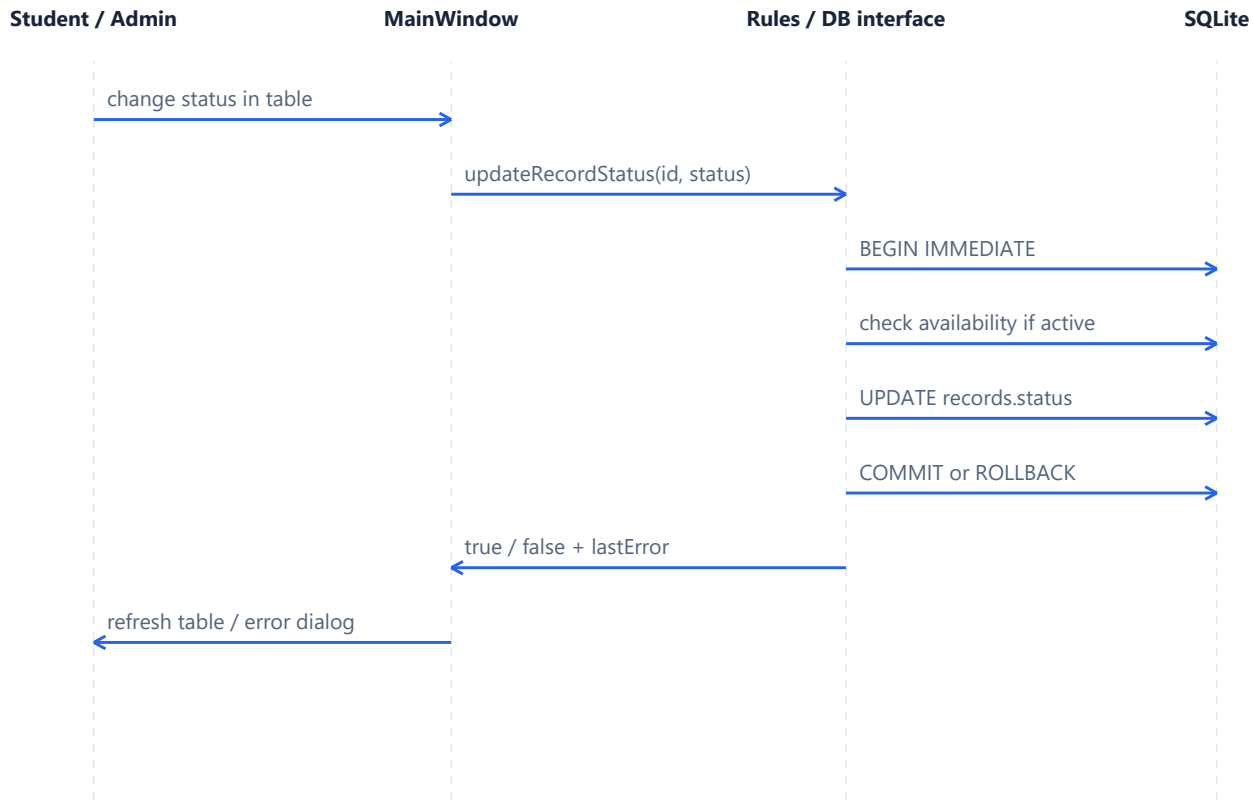

## Transaction outcome

The window validates input and prepares a new Pending record. The database transaction checks key/slot availability, saves related facts, inserts the booking and replaces child slots. A failure rolls back the transaction and returns lastError. A successful save shows the allocation key.

Sequence reflects active submitData(), AllocationRules and SqliteDatabaseManager::addRecord().

CRITICAL FLOW / 2

# Sequence: change approval status




## Conflict-aware review

The administrator changes the row status. The database validates the new value and rechecks slots before moving a rejected booking to an active state. If occupied, it rolls back. The window restores the previous display value and shows the error. Rejected records release availability while remaining visible in the record list.

UI Approved/Rejected labels map to stored Approve/Reject values. Pending also reserves its selected slots.